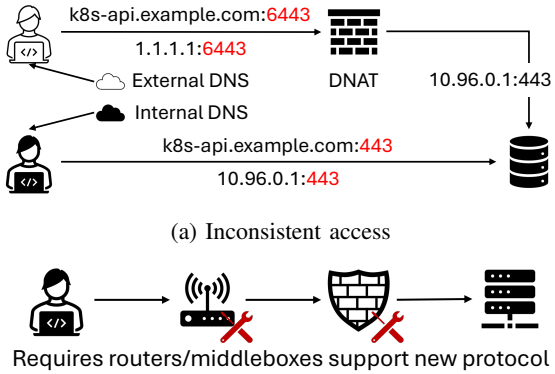
POND: Accelerating Cross-Realm Service Access with Transparent and Name-based Packet Delivery

***Abstract*—In cross-realm service access scenarios, overlapping IP address spaces frequently cause routing conflicts. Traditional IP-based solutions like tunnels, NAT, and overlays often necessitate complex Split-Horizon DNS infrastructure that fails to address port mismatches between internal and external networks. Moreover, some of these approaches require intrusive agent deployment. While name-based routing offers a theoretical solution, existing implementations similarly fail to resolve such port inconsistencies, mandate impractical infrastructure upgrades, or suffer from the performance bottlenecks of user-space processing. To address these limitations, we propose POND, a transparent, high-performance name-based routing system. Unlike prior approaches, POND operates entirely within the Linux kernel, leveraging TLS SNI to identify services without modifying client applications, network stacks or legacy middleboxes. POND employs a novel 4-way TCP-TLS joint handshake proxy to intercept TLS connections and a deferred in-kernel name resolution mechanism to route packets at wire speed, effectively eliminating the context-switching overhead of user-space proxies. Evaluation results demonstrate that POND outperforms the widely deployed user-space proxy, HAProxy, improving throughput and latency by 33.74% and 33.92%, respectively, while maintaining high in-kernel name-rules scalability.**

***Index Terms*—cross-realm service access, name-based routing, netfilter, deferred in-kernel name resolution**

I. Introduction

With the rapid evolution of the Internet, cross-realm service access has become a cornerstone of modern cloud infrastructures. While certain public services are globally reachable, a wide variety of critical resources, ranging from corporate databases to internal administrative tools, reside within isolated private networks (intranets). Scenarios such as remote telecommuting, cross-organizational collaboration, and multi-cloud resource orchestration [1], [2] render the ability to access these distributed intranet resources indispensable. Typically, these intranets operate within dedicated private IP address spaces [3] and demand secure access from the Internet, usually achieved using tunnels (e.g., WireGuard [4]). However, this widespread deployment model creates a pervasive challenge: *IP address space overlap*. Since private IP ranges are reused universally, address collisions are inevitable when a client attempts to connect to multiple independent intranets simultaneously. This overlap fundamentally breaks the global uniqueness assumption of IP-based routing, transforming what should be seamless connectivity into a complex addressing puzzle. When two realms utilize the same or overlapping private IP subnets, policy-based routing cannot unambiguously determine which tunnel interface should transmit a packet destined for a conflicting IP.



(a) Inconsistent access

(b) Incompatibility with legacy routers and middleboxes

(c) Performance overhead of user-space proxies

Fig. 1: Limitations of current solutions to IP address space overlap

To address this issue, various solutions and tools have been proposed. We categorize them into IP-based and name-based approaches. NAT-based routing [5] and overlay-based routing [6] are common IP-based solutions. However, NAT-based approaches necessitate a complex "Split-Horizon DNS" infrastructure [7]. They also frequently introduce port mismatches between external and internal service endpoints. Meanwhile, overlay-based approaches incur high deployment costs since they require specialized agents on every node. In addition, these approaches necessitate maintaining a global IP Address Management (IPAM) system to assign a unique virtual IP address to each node, which represents a non-negligible extra overhead.

Given these shortcomings, *name-based routing* emerges as a compelling alternative. By elevating routing decisions from the conflicting L3 IP space to the higher level name space, services can be uniquely identified by their names, theoretically bypassing IP collisions. Names offer a virtually infinite namespace and can carry semantic information that facilitates intelligent routing without global IP coordination.

However, existing name-based routing solutions also ex-

hibit the following three critical drawbacks. **Inconsistent Access Experience**. Existing solutions often fail to provide a seamless and unified access experience across Internet and intranet environments. By relying on complex split-horizon DNS configurations to distinguish between internal and external access, these solutions fragment the service namespace. Consequently, clients are unable to access resources using the same name consistently across different network locations, as shown in Fig. 1a. **Compatibility issues**. Fig. 1b shows that academic proposals such as IPNL [8] and HIP [9] typically require modifications to the host protocol stack or introduce new packet headers. The latter mandates that all on-path middleboxes support the new protocol, incurring prohibitive infrastructure upgrade costs. **Performance overhead**. Widely deployed application-layer proxies (e.g., HAProxy [10], Nginx [11]) operate in user space, incurring overhead due to frequent context switches and memory copying. This results in lower throughput and higher latency compared to in-kernel packet forwarding, shown in Fig. 1c.

To address these challenges, this paper proposes the **P**acket **O**ver **N**ame-based **D**elivery (**POND**) architecture. It consists of the following three primary technologies.

To ensure a consistent access experience, we propose a cross-layer name-based routing scheme that utilizes the combination of the TLS server name and TCP port as the service identifier. This identifier is automatically extracted within the Linux kernel's netfilter framework by intercepting TCP and TLS handshake packets. Consequently, users are relieved from the burden of learning new names, enabling them to access services using uniform identifiers across both internal and external networks. **To address the compatibility challenges**, we propose a 4-way TCP-TLS joint handshake proxy model. Leveraging the existing network stack, this model operates transparently to both clients and network infrastructure. It performs routing decisions based on service identifiers while employing connection tracking to maintain per-stream context. To execute these decisions, the mechanism rewrites destination IP addresses and TCP ports, seamlessly forwarding the traffic to the intended target. **To minimize performance overhead**, POND introduces a deferred in-kernel name resolution mechanism. Instead of performing name resolution in user-space, this mechanism defers it to the moment immediately preceding the routing decision (just-in-time). To achieve this, we provide a new `iptables/nftables` target extension, `PONDPROXY`, which enables user-space programs to dynamically add or remove name-to-destination address rules into the kernel. Consequently, name resolution is seamlessly integrated into the routing process, thereby maximizing packet-forwarding performance.

The main contributions of this paper are as follows:

- The **cross-layer name-based routing scheme** that utilizes TLS SNI to decouple service identity from network location, ensuring consistent access across overlapping address spaces without modifying current network architecture.
- The **4-way TCP-TLS joint handshake proxy model** within the Linux kernel, which transparently intercepts connections and extracts service names, avoiding the performance overhead of context switching inherent in user-space proxies.
- The **deferred in-kernel name resolution mechanism** supported by a custom radix-tree implementation, which enables efficient, scalable name-to-address translation at routing time.

We validate the functionality of the POND system in a cross-domain environment. We also conduct experiments to evaluate its performance. Results show that POND improves throughput and latency by 33.74% and 33.92%, respectively, compared to the widely deployed solution.

II. Background and Motivation

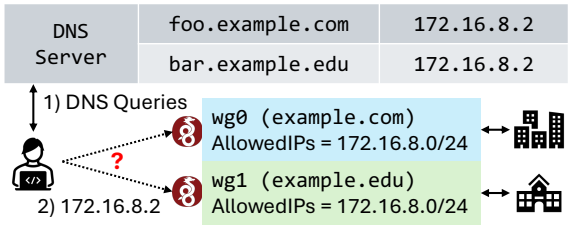
A. The Persistence of Address Space Overlap

Address space overlap persists as a pervasive challenge in modern networking. In scenarios such as mergers and acquisitions (M&A) or multi-cloud deployments, interconnecting networks with identical private address ranges inevitably leads to routing ambiguity, while renumbering these networks is often operationally prohibitive. This limitation is particularly evident in the hub-and-spoke architectures offered by major cloud providers. For instance, services like AWS Transit Gateway [1] and Azure Virtual WAN [2] explicitly mandate that attached Virtual Private Clouds (VPCs) utilize non-overlapping CIDR blocks, thereby highlighting the friction between legacy network constraints and modern cloud integration.

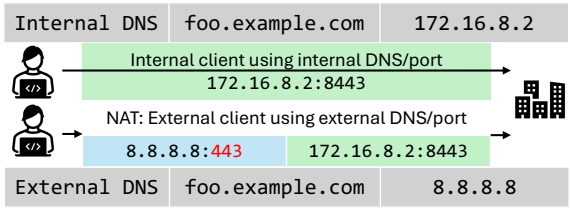
The transition to IPv6 does not fully resolve these address overlapping issues due to two primary factors. Firstly, entrenched IPv4 usage: IPv6 adoption remains slower than anticipated [12], while private IPv4 addresses [3] remain deeply embedded in enterprise, cloud, and ISP environments [13]. Second, persistent translation paradigms: Many enterprises employ NPTv6 [14] combined with Unique Local Addresses (ULA) to avoid vendor lock-in. This practice mirrors the NAT44 architecture, thereby re-introducing the semantic gap between internal and external addressing and leading to potential prefix collisions.

B. Limitations of IP-Based Solutions

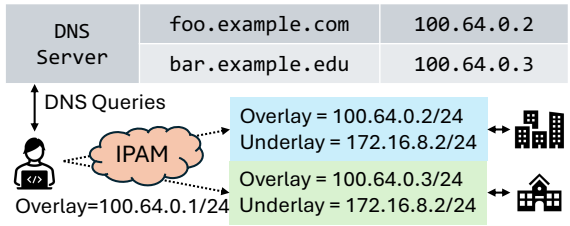
To handle cross-realm access where address spaces may collide, existing solutions typically adopt one of three models: *tunnel-based*, *NAT-based*, or *overlay-based* routing. In the *tunnel-based* approach (e.g., WireGuard [4]), clients maintain simultaneous tunnels to different realms. However, when two realms utilize the same or overlapping private IP subnets, policy-based routing cannot unambiguously determine which tunnel interface should transmit a packet destined for a conflicting IP (as shown in Fig. 2a), resulting in routing failures. The *NAT-based* approach (Fig. 2b) avoids direct IP conflicts by projecting internal services to public endpoints. Nevertheless, it necessitates a complex "Split-Horizon DNS [7]" infrastructure to provide distinct IP resolutions for internal and external



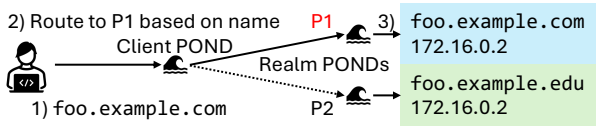
(a) IP-based routing in tunnel.



(b) IP-based routing in NAT.



(c) IP-based routing in Overlay.



(d) Name-based routing in POND.

Fig. 2: Limitations of IP-based routing compared to POND

clients, significantly increasing maintenance overhead. Moreover, NAT creates a semantic gap between the L3 addressing provided by DNS and the actual L4 service endpoint. Since NAT often employs L4 port forwarding, the external service port may differ from the internal one. As standard DNS cannot convey this port discrepancy, external clients are left unable to discover the correct service entry point without fragile workarounds [15]. *Overlay-based* approaches (e.g., VXLAN [16]) attempt to bypass conflicts by constructing a unified virtual IP layer (Fig. 2c). While conceptually straightforward, this method is intrusive and resource-intensive: it requires deploying specialized agents on every node and establishing a global IP Address Management (IPAM) system to coordinate the virtual space. Consequently, the high deployment cost and complexity make it unsuitable for lightweight or unmanaged access scenarios.

C. Limitations of Existing Name-Based Solutions

Existing name-based solutions generally fall into two categories: clean-slate academic proposals and practical application-layer proxies. Both approaches face significant limitations in delivering transparent and high-performance cross-realm access.

Protocol incompatibility in academic proposals. Academic works such as IPNL [8] and HIP [9] introduce new layers or headers to decouple identifiers from locators. IPNL requires a new L3.5 header, necessitating modifications to the host OS network stack. HIP introduces a Host Identity Protocol layer, which requires not only host modifications but also upgrades to network middleboxes (firewalls, NATs) to recognize and pass the new protocol. These requirements create a high barrier to entry, as upgrading the entire infrastructure is often infeasible.

Limitations of User-Space Proxies. In practice, name-based routing is typically achieved via application-layer proxies like HAProxy [10] and Nginx [11]. However, these solutions suffer from both performance and deployment drawbacks. Operationally, they function in user space, terminating connections to extract service names. This forces data to traverse the user-kernel boundary twice, incurring significant overhead from context switching and memory copying, which results in reduced throughput and increased latency. Structurally, they often fail to provide transparent access. Administrators must rely on complex "Split-Horizon DNS" [7] or explicit client-side configurations to route traffic to the proxy, increasing management complexity and potentially breaking the unified namespace experience.

D. The opportunity of POND

Given the shortcomings, the underlying causes of IP address space overlap and the difficulties in resolving it can be attributed to the following three factors:

- **IP address is not the ideal service identifier.** As noted above, services are typically identified by a L4 3-tuple (TCP/UDP), and relying on a L3 address (IP) for service identification and request routing will lead to inconsistency. Likewise, if names were to serve as service identifiers, the existing DNS, which merely resolves names to L3 addresses, would be inadequate.
- **Insufficient routing information.** Routing systems typically rely only on packet information available at L3 layer (e.g., policy-based routing) or L4 layer (e.g., netfilter) to make routing decisions. These systems cannot access information above the TCP/UDP 5-tuple and therefore unable to make routing decisions only based on conflicting routing rules or IP addresses.
- **High compatibility requirements.** Modifying the host protocol stack or upgrading network infrastructure (e.g., routers, firewalls) to support new protocols is often cost-prohibitive and operationally infeasible. Therefore, compatibility with existing protocols and network infrastructure is crucial for any practical solution.

To address these challenges effectively, a solution must satisfy the following three requirements:

- **Identity-Locator Separation:** It should use service names (which are unique and contain semantic informa-

tion) rather than IP addresses (which are opaque locators) for routing decisions.
- **Transparency:** It should work with unmodified client applications and standard TLS+TCP/IP stacks. Meanwhile, it should not introduce new components like global naming coordinators.
- **Performance:** It should operate at the kernel level to minimize overhead, matching the efficiency of traditional packet forwarding.

POND meets these needs by leveraging TLS SNI to make routing decisions within the kernel space. Fig. 2d shows the basic idea of our proposed POND system. When a user issues a request for `foo.example.com`, the client-side POND proxy (in-kernel and transparent to the client applications) establishes the TLS session with the client. Then, based on the extracted service name, POND proxy routes it to the appropriate realm-side POND instance (P1, also in-kernel). P1 then performs its own name-based lookup and forwards the connection to the designated real server.

III. Related Work

Evolution of Name-based Routing. Early proposals like IPNL [8] and HIP [9] aimed to decouple service identifiers from locators but required invasive changes to host protocol stacks or network middleboxes. Clean-slate architectures like ICN [17] and NDN [19] replace the IP model entirely, hindering deployment. POND achieves similar decoupling using ubiquitous TLS SNI without requiring any infrastructure upgrades.

IP-Centric Solutions. Tunneling (e.g., WireGuard [4]) and Overlay networks (e.g., VXLAN [16]) attempt to manage address spaces but struggle with overlapping subnets or require complex global IPAM systems. IPv6 [18] alleviates address exhaustion but faces slow adoption and potential prefix collisions in NAT66 scenarios. POND bypasses these IP-layer constraints by routing on names.

Application-Layer Proxies. User-space proxies (e.g., HAProxy [10], Nginx [11]) provide name-based routing but incur significant performance overhead from user-kernel context switches and often require Split-Horizon DNS [7]. Recent optimization efforts [20], [21] often focus on specific niches rather than general-purpose routing. POND moves the proxying logic into the kernel to combine the flexibility of names with the performance of L4 forwarding.

Table I summarizes the differences between POND and existing work. POND distinguishes itself by achieving transparent, high-performance name-based routing without requiring modifications to the host protocol stack or current network infrastructure.

IV. POND Design

A. Overview

The name-based routing design in POND consists of three main components: name selection, connection proxying, and name resolution.

POND focuses specifically on TLS traffic, utilizing the server name and port (hereinafter referred to as the *server name*) derived from the TLS Server Name Indication (SNI) extension to identify each connection. HTTPS (including TLS, TCP, and IP) is forming the new narrow waist of today's Internet architecture [22]. As of July 2025, HTTPS accounts for 96% of all web traffic [23], and 75.3% of websites support modern TLS 1.3 [24], demonstrating the broad applicability of name-based routing for TLS flows. The TLS Server Name Indicator (SNI) was originally introduced to present the correct certificate when hosting multiple sites on a single host [25], but it conveniently doubles as a service identifier.

To proxy TLS traffic and extract the server name, we propose the **4-way TCP–TLS joint handshake proxy**. Since the TLS SNI is carried in the `Client Hello` packet which is the very first packet right after TCP handshake, the proxy cannot decide where to forward the incoming TCP flow until it has received the `Client Hello`. Therefore, our proxy should suspend the incoming TCP handshake with the client, extract the `Client Hello` to read the SNI, and only then initiate a fresh handshake towards the real server.

To efficiently redirect the connection to a specified real server by name, we introduce a **deferred in-kernel name resolution** mechanism. Unlike traditional approaches where name resolution occurs in user space prior to connection establishment, POND defers this process to the kernel's packet forwarding path. By maintaining name-to-address mappings in an efficient in-kernel data structure, POND resolves the destination IP address immediately after extracting the service name. This in-kernel design eliminates the performance penalty of user-space lookups and ensures that routing decisions are made after kernel-space name lookups just-in-time, effectively resolving ambiguities caused by IP address space overlap.

Fig. 3 shows a typical cross-realm service access scenario enabled by POND. A client issues a request for a service in a remote realm whose FQDN is `example.com` and whose real IP is 172.16.0.2. When the client-side POND proxy intercepts this request, it establishes a 4-way-handshake proxy session, parses the TLS SNI to get server name, and performs routing-time name resolution to rewrite the destination to the Internet access point of the target realm. Upon arrival at the realm-side POND proxy, the same proxying and name-resolution logic is applied, this time rewriting the destination to the real server's intranet address. When the real server processes the packet and generates a response, the two tiers of POND proxies (realm and client) apply their SNAT rules in reverse, returning the packet transparently to the client. Since the real server's intranet address is never exposed to either the client or the client-side proxy, overlapping IP address spaces between realms do not interfere with correct packet routing. For domains subject to POND forwarding, the user-space DNS must always return a result—regardless of its correctness—to ensure the request reaches the kernel for processing; for domains that do not match the kernel name-rule set, POND performs no rewriting and transmits packets as is.

TABLE I: Comparison of Related Works

Scheme	Identifier	Routing Layer	Deployment Requirements	Key Considerations
IPNL [8]	FQDN	L3 (Modified)	Packet headers & infrastructure	Requires host OS network stack upgrades
HIP [9]	Host Identity	L3.5	Host HIP protocol stack	Requires network infrastructure (middleboxes) upgrades
ICN [17]	Content Name	Clean-slate	Network architecture	Incompatible with existing IP infrastructures
Tunneling [4]	IP Address	L3	Client endpoint configuration	Routing ambiguity on address space overlap
Overlay [16]	Virtual IP Address	L3 (Virtual)	Overlay software/agent	Global IPAM/Inaccessible to non-overlay services
IPv6 [18]	IPv6 Address	L3	Network infrastructure	High migration cost & Potential ULA conflicts in NAT66
App Proxy [10]	Service Name	L7 (User-space)	User-space proxy application	High context-switch overhead & Requires Split DNS
POND	**TLS SNI**	**L6 (Kernel)**	**Linux kernel module**	**Transparent, Compatible & Low Overhead**

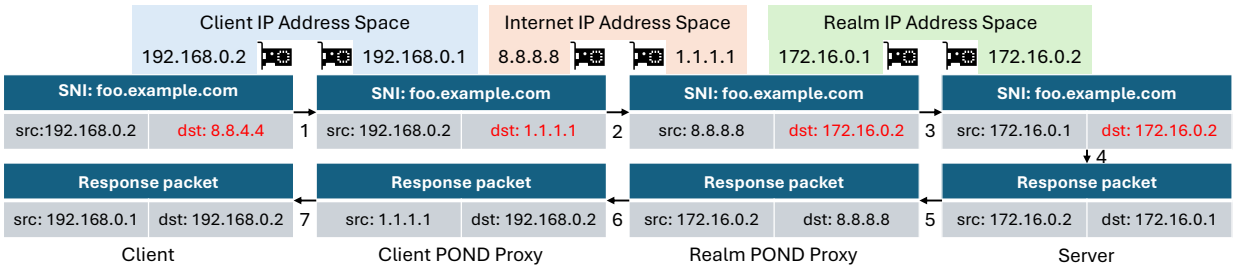


Fig. 3: An example of a client using POND for cross-realm service access. When the client initiates a TLS connection, the client POND proxy intercepts the stream, extracts the TLS SNI, and rewrites the destination to the designated realm POND proxy. The realm proxy performs a similar intercept-extract-rewrite process and forwards the stream to the target server. Response packets traverse the reverse path. POND makes address space overlaps between realms transparent to the client.

B. 4-way TCP-TLS Joint Handshake Proxy

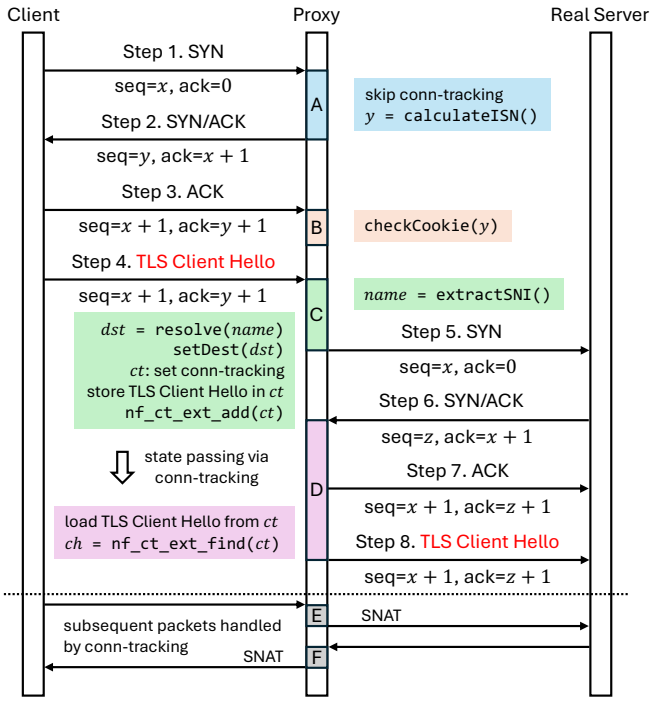


Fig. 4: Sequence diagram of the proposed 4-way TCP-TLS joint handshake proxy.

We propose a 4-way TCP-TLS joint handshake proxy to enable in-kernel, name-based routing, as shown in Fig. 4. The 4-way handshake is implemented as an extension of kernel netfilter [26]. Our 4-way handshake can be divided into four handshake stages, labeled A, B, C, and D, in addition to two post-handshake SNAT stages, E and F. The proposed 4-way handshake can be applied to the `filter` table of the `FORWARDING` chain on the proxy server, as well as to the `filter` table of the `OUTPUT` chain on the client, as shown in Fig. 5. This section only describes the proxy server scenario for brevity; the client-side scenario follows the same principles.

Stage A is `SYN` hijacking. During the TCP 3-way handshake, the proxy has not yet read the name and thus cannot determine where to route the `SYN` packet. Therefore, it is necessary to temporarily steal (`NF_STOLEN`) the `SYN` packet. Similar to the `SYNPROXY` design [27], our `PONDPROXY` kernel module fully manages the entire handshake process. To facilitate this, connection tracking is disabled for all inbound `SYN` packets. This prevents the connection tracking subsystem from preempting the proxy's control over the handshake. Due to the absence of connection tracking entry, the TCP 3-way handshake should be stateless. Thus, the proxy computes the Initial Sequence Number (ISN) according to the SYN Cookie algorithm and sends the `SYN/ACK` response to the client.

Stage B is `ACK` verification. Since the TCP handshake process is stateless, the proxy cannot retrieve information from the connection tracking entries for the first two steps of the handshake. Instead, it needs to extract the SYN Cookie set in Stage A from the `ack` number of the `ACK` packet and verify whether it was issued by itself. This mechanism can also be used to defend against `SYN` flooding attacks. At this point, the proxy has completed the TCP three-way handshake with the client, but still needs to wait for the client to send the TLS handshake packet before it can proceed with the handshake

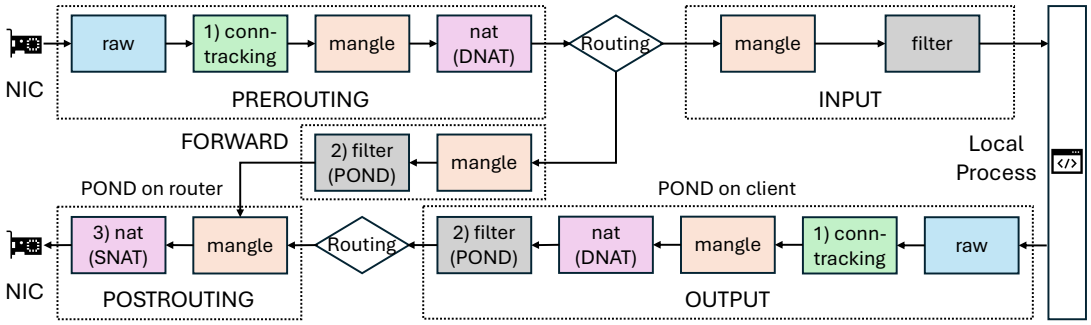


Fig. 5: Packet flow diagram of kernel netfilter. PONDPROXY(shortened as POND) takes effect at `filter` table in `FORWARD` or `OUTPUT` chain.

with the real server.

Stage C is name extraction and destination determination. For TLS requests, after the TCP handshake is completed, the client proceeds with the TLS handshake, whose very first packet is the `Client Hello`. Upon receiving the `Client Hello`, the proxy should extract the server name from the SNI extension. Then, it checks whether this server name matches any entry in the name-rule set, thereby determining the destination address for the current connection. Finally, the proxy sends a `SYN` packet to the determined destination address to initiate the TCP handshake with the real server.

It is worth noting that, when the proxy sends the `SYN` packet to the real server, it must simultaneously set up a connection tracking entry. This is necessary because traffic after the 4-way handshake still needs to be handled by connection tracking, so establishing the connection tracking entry as soon as the destination address is determined is the appropriate timing. Additionally, unlike the `SYN/ACK` packet sent by the proxy to the client (Step 1 in Fig 4), the `SYN` packet sent by the proxy to the real server (Step 5 in Fig 4) can no longer carry any state information. The reason is that the sequence number of the `SYN` packet in Step 5 must remain consistent with that of the `SYN` packet sent from the client to the proxy in Step 1, to ensure that subsequent TCP sequence numbers between client and server are synchronized. Since the `SYN` packet in Step 5 cannot include extra information, the proxy will store the `Client Hello` information statefully in the connection tracking entry for later recovery.

Stage D is `ACK` and TLS `Client Hello` issuing. After the proxy receives the `SYN/ACK` from the real server, it should respond an `ACK` packet to the real server to complete the TCP handshake. Since the `SYN` packet in Step 5 and the `SYN/ACK` packet in Step 6 belong to the same connection tracking entry, the proxy can extract the information stored during stage C that is necessary for reconstructing the `Client Hello` packet, and send the proxied `Client Hello` message to the real server. After this step, the TCP-TLS joint handshake is completed, and subsequent traffic will be managed by connection tracking.

Stage E and F performs `SNAT`. From the perspective of connection tracking, the 4-way handshake process essentially performs a manual `DNAT` operation. To ensure that packets can be correctly routed to the real server behind the proxy, whether on the same or a different network, it is necessary to conduct an `SNAT/MASQUERADE` operation on both outgoing and incoming packets, respectively. As shown in the Fig. 5, `SNAT` always takes place in the `nat` table of `POSTROUTING` chain, after POND taking effect at `filter` table. For outgoing packets originating from the client, `SNAT` sets the source IP address of the packet to the proxy's IP address on the interface facing the real server. For incoming packets from the real server, `SNAT` sets the source IP address of the packet to the proxy's IP address on the interface facing the client. It can be readily observed that `PONDPROXY` is always attached to the `filter` table rather than to the `nat` table normally used for `DNAT` operations. This is because `PONDPROXY` should invoke `ip_route_me_harder()` and `ip_local_out()` to issue new packets, a functionality that the `nat` table does not provide.

State maintenance in the 4-way handshake. The handshake process entails two distinct types of state maintenance. The state passing from stage A to stage B is realized using SYN cookies, so the proxy does not need to maintain any per-connection state internally. By contrast, the state passing from stage C to stage D relies on an extension to the connection-tracking records. At stage C, the proxy must persist all the necessary information required later, at stage D, to send both the `ACK` and a faithful `Client Hello` to the real server. Issuing the `ACK` is straightforward, but reconstructing the `Client Hello` involves preserving the SNI extension and the client's TLS fingerprint [28] [29]. Because different clients support different TLS cipher suites and other parameters, the proxy must reproduce these characteristics exactly when sending the `Client Hello` to real server in stage D to avoid handshake failures due to fingerprint mismatches. To achieve this, we augment the connection-tracking record with a custom extension that stores the requested SNI and an index to the corresponding TLS-fingerprint entry.

C. Deferred In-Kernel Name Resolution

The 4-way TCP–TLS joint handshake proxy enables kernel-level name-based routing. Beyond merely proxying connec-

tions, POND also needs to resolve names into specific destination IP addresses at routing time, which is called Deferred In-Kernel Name Resolution (DKNR). In this design, *deferred* signifies that the binding between a service name and its destination IP is determined at the exact moment of packet routing, rather than being pre-resolved at user-space. *In-kernel* indicates that this look-up process executes entirely within the kernel's packet processing path. In the particular scenario of cross-realm resource access, the name resolution system of POND focuses on the following two types of rules: 1) Mapping a particular domain name to a specific IP address; 2) Mapping all domain names that share a given suffix to a specific IP address. Note that this "suffix" is not necessarily a Top-Level Domain (TLD) but a literal string suffix. Such suffix matching enables POND to route all connections belonging to a particular realm to the same IP address.

To accommodate both exact-match and suffix-match requirements, POND employs a specially designed radix tree to store and query domain rule—destination IP records. In order to ensure that matching always starts from the beginning of the string, we use reversed Fully Qualified Domain Name (FQDN) and reversed name rules during the matching process, which means reversing the order of domain name labels. For a more rigorous formalization of the radix tree of POND, we summarize the notation and terminology in Table II.

TABLE II: Notation list of POND Radix Tree

Symbol	Definition
`nil`	Null character.
$\star$	Wildcard character. $\star$ can match any non-`nil` prefix of an FQDN[a]. It can only appear as the very first label in a name rule and may appear at most once.
Σ	FQDN alphabet, with label characters and separator dot (`.`). Label characters include Letters (case-insensitive), Digits and Hyphen (LDH). $\Sigma = \{\texttt{a},\ldots,\texttt{z}\} \cup \{\texttt{0},\ldots,\texttt{9}\} \cup \{\texttt{-}\} \cup \{\texttt{.}\}$.
$\lvert\Sigma\rvert$	Size of the FQDN alphabet Σ. $\lvert\Sigma\rvert = 38$.
Σ^*	Set of all FQDNs formed using characters from the alphabet Σ with a maximum length of 255.
$\sigma \in \Sigma^*$	A queried FQDN.
Π	Name rule alphabet. Adding $\star$ to Σ to enable suffix matching. $\Pi = \Sigma \cup \{\star\}$.
$\lvert\Pi\rvert$	Size of the name rule alphabet Π. $\lvert\Pi\rvert = 39$.
$\pi \in \Pi^*$	A name rule.
Π^*	Set of all name rules formed using characters from the alphabet Π.
$\pi \sqsupset \sigma$	Name rule π is a *suffix* of FQDN σ.
$\pi \sqsubset \sigma$	Reversed name rule π is a *prefix* of reversed FQDN σ.
T_r^n	Pond radix tree storing the name rules, with r as the radix and n leaves.

[a]Unlike DNS names where wildcards can only match a single label.

In POND radix tree, each internal node maintains the following three fields:

- `offset`: For the queried FQDN, `offset` specifies which character index to examine when selecting the next child branch. As the lookup moves deeper into the tree, the `offset` value always increases.
- `bitmap`: A fixed-length `bitmap` that precisely indicates which outgoing branches (i.e., which characters in $\Pi \cup \{\texttt{nil}\}$) are present at this node. `bitmap` has a size of 40, in which each bit corresponds to one character in $\Pi \cup \{\texttt{nil}\}$. If there is a branch for symbol $c \in \Pi \cup \{\texttt{nil}\}$, the bit associated with c is set to 1.
- `children`: A pointer to the array of child-node references. Children nodes are laid out in the same order as the 1-bits in `bitmap` (i.e. in increasing bit-index order).

Each leaf node maintains the following three fields:

- `key`: The name rule π, which may be a reversed FQDN or a reversed FQDN-suffix pattern. `key` $\in \Pi$.
- `value`: The destination IP address associated with the name rule.
- `match`: The `match` type, classified into one of the three categories: 1) exact match: the `key` of the leaf is a reversed FQDN. We check whether the `key` exactly equals the queried FQDN; 2) prefix match: the `key` of the leaf is a prefix of a reversed FQDN (i.e. a suffix of a FQDN). We check whether the `key` is a suffix of the queried FQDN; 3) skip match: the leaf is reached by traversing a wildcard ('$\star$') branch in its parent `bitmap`; since the parent's wildcard matches any sequence of characters, we skip matching and directly return the value.

Fig. 6 shows an example of a 6-rule POND radix tree and the corresponding matching process. It is straightforward to show that the POND radix tree satisfies the following properties: $\forall \sigma \in \Sigma^*$:

- if $\forall \pi \in T$, π cannot match σ, the query algorithm returns `NULL`;
- if $\exists \pi \in T$, π matches σ, the query algorithm returns the destination IP of the name rule π^* with the highest priority. The priority order follows: exact matches outrank all others, and among all non-exact matches those with longer prefixes take precedence.

We analyze the time and space complexity of queries in the POND radix tree. Suppose that each queried FQDN is $\sigma \in \Sigma^*$, where $|\sigma| = m$. Let radix tree T_r^n contains n name rules (i.e. n leaves). A query proceeds from the root node down through zero or more internal nodes and finally arrives at exactly one leaf node. At each internal node, reading the character $\sigma[\texttt{offset}]$ is $O(1)$. Using the `bitmap` of the node to select the next child pointer is also $O(1)$. For an exact-match or prefix-match leaf, verifying that σ matches the stored `key` takes $O(m)$. For a skip-match leaf, the match test is $O(1)$. If the final leaf node has a depth of d, the total time per query is $O(d+m)$. In an r-ary radix tree T_r^n with n leaves, the depth d satisfies

$$\Omega(\log_r n) \le d \le O(n)$$

Hence the overall time complexity per query admits the following bounds: best (balanced) case: $\Omega(\log_r n + m)$, worst (degenerate) case: $O(n+m)$. It is worth noting that in the POND radix tree the radix $r = 40$, which is relatively large, so for typical rule-set sizes n the value of $\log_r(n)$ changes only marginally. This implies that the POND radix tree scales

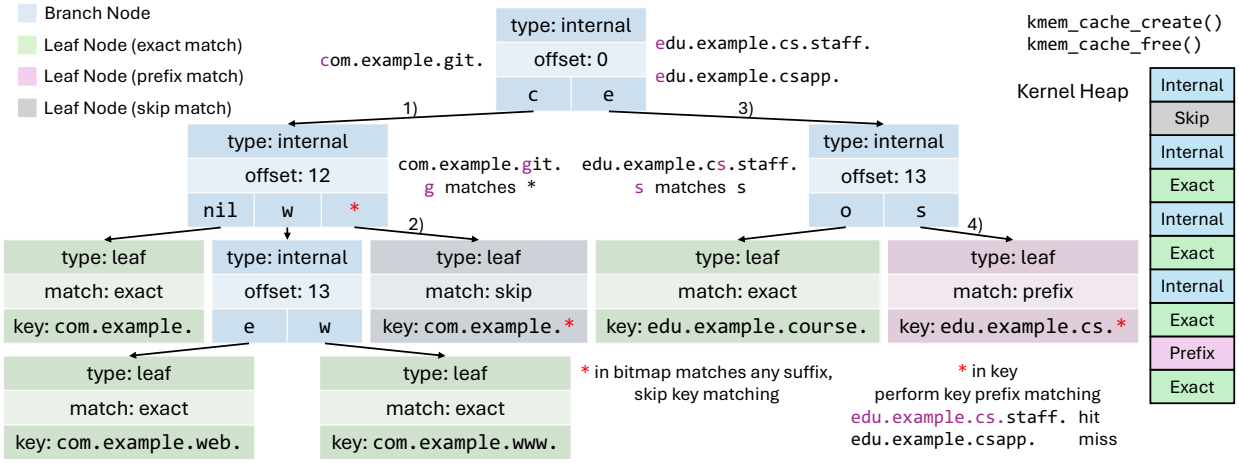


Fig. 6: A POND radix tree with 6 name rules: 4 exact-match rules and 2 prefix-match rules. The following two examples illustrate two possible scenarios for the wildcard $\star$ in the radix tree. When querying the reversed FQDN `com.example.git.`, the traversal reaches the second internal node and the next character `g` matches $\star$, causing a jump into a leaf node of skip match for direct return. When querying `com.example.cs.staff.`, the traversal at the second internal node enters a leaf node of prefix match to perform the prefix checking.

well as the number of rules n increases. We will examine this issue in detail in Section V-B.

The space complexity of POND radix tree is given by the total space occupied by all leaves and all internal nodes. Let b denote the number of internal nodes, each internal node I_i has out-degree $\deg(I_i)$. The total out-degree of the tree is $\Sigma_i \deg(I_i)$, which equals the total number of edges $b+n-1$. $\Sigma_i \deg(I_i)$ satisfies $2b \leq \Sigma_i \deg(I_i) \leq \sigma b$. So

$$\frac{n-1}{\sigma-1} \leq b \leq n-1$$

Therefore, the space complexity of the POND radix tree is $\Theta(n)$. We will verify the result in Section V-B.

In POND, the nodes of radix-tree are allocated in the kernel heap. To let user-space programs manage these nodes, POND provides an xtables extension called `PONDPROXY` that invokes `kmem_cache_alloc()` and `kmem_cache_free()` to call into the slab allocator of kernel for memory allocation and deallocation. User-space applications add or remove name-based rules simply by calling the `PONDPROXY` extension. From the perspective of a user-space program, managing the name rules of POND is just as easy as managing DNAT rules in `iptables`. The only difference is that the matching parameter has been changed from a destination IP address to a name rule.

D. Implementation

We implement the POND system on a Linux server equipped with Intel(R) Xeon(R) Gold 6442Y 48-core CPU and 256 GB of RAM, running Arch Linux (kernel version `v6.18.5-arch1`) and iptables v1.8.11 (`nf_tables`). The entire system is written in C. In kernel space, $\sim$ 1.5K lines of C code handle the 4-way TCP-TLS joint handshake and $\sim$ 1.2K lines of C code implement the radix-tree name matching; in user space, $\sim$ 0.5K lines of C code realize the new `xtables` target extension.

V. Evaluation

POND is designed to ensure transparent access and architectural compatibility within complex networking environments. To validate these attributes, we deployed POND in a real-world, multi-regional setting spanning two organizations with overlapping IP address spaces. In this deployment, POND enables users to access services (e.g., Kubernetes API servers) via a unified service name from both intranet and Internet environments. Crucially, it allows a single client to concurrently access conflicting IP address spaces across regions without requiring manual network switching.

While prioritizing functional flexibility, we also aim to demonstrate that POND's architectural innovations do not compromise efficiency. In this section, we conduct a series of experiments to evaluate POND's performance by addressing the following questions:

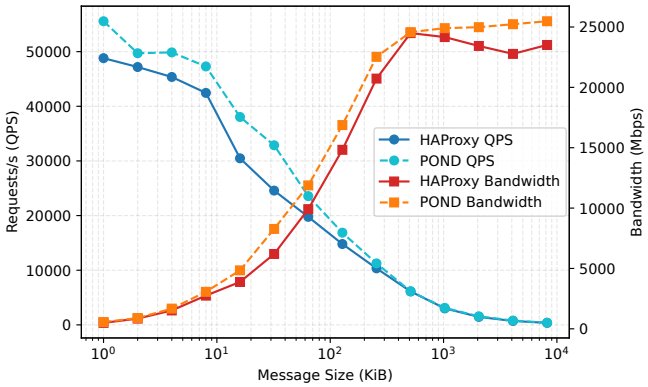
- How does POND's in-kernel 4-way TCP-TLS joint handshake proxy compare to existing user-space L4 proxy in terms of data forwarding throughput and QPS?
- To what extent does POND reduce the end-to-end latency for TLS-based service access compared to user-space proxies across different message sizes?
- How does the deferred in-kernel name resolution mechanism of POND scale in terms of memory usage and lookup overhead as the number of name rules increases?

A. Testbed

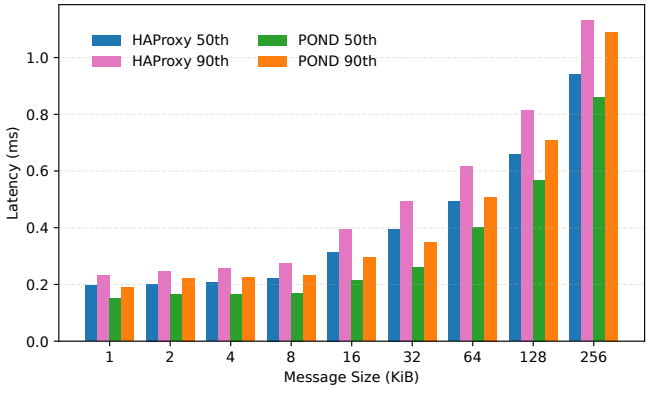
Our testbed consists of a physical server with 48 CPU cores and 256 GB of memory, together with multiple KVM virtual machines each provisioned with 16 cores and 32 GB of memory. On the physical server we deploy either our POND

system or, for comparison, a user-space L4 load balancer; in both cases it serves as a proxy that routes incoming requests to specified destinations. We use one 16-core KVM VM as the client to generate HTTPS traffic, and ten additional 16-core KVM VMs to emulate 800 real servers, each performing TLS termination and processing HTTPS requests. The client–proxy and server–proxy links are provided by Mellanox ConnectX-6 NICs operating at Ethernet mode with a bitrate of 200 Gbps. As the backend service responsible for TLS termination and request handling, we employ nginx [11], serving static files of varying sizes to emulate real-world traffic patterns. For a user-space routing baseline, we choose HAProxy, owing to its wide adoption and proven high performance. We use the server certificate issued by Google Trust Services that employs an ECDSA key on the secp256r1 curve. It is worth noting that although we use HAProxy as a comparative baseline, HAProxy itself does not resolve the issue of IP address-space overlap; it merely performs user-space DNAT based on names.

B. Performance



(a) Throughput and QPS comparison with different message sizes.



(b) Latency comparison with different message sizes.

Fig. 7: End-to-end throughput and latency comparison with different message sizes.

End-to-end Throughput. We evaluated the end-to-end throughput of the POND system and HAProxy under different message sizes. We use *wrk* [30] to generate HTTPS requests, and use nginx to serve pre-generated random files of different sizes. Clients request these files via different URLs, thereby emulating real-world traffic. L4 load balancing is performed on either POND or HAProxy, using the TLS SNI extension for name-based routing. In POND, we apply the 4-way TCP-TLS joint handshake proxy introduced in Section IV-B to extract the SNI, and then rewrite the destination IP address via deferred name resolution proposed in Section IV-C. In HAProxy, traffic is processed in user-space, where we use the `req.ssl_sni` ACL to extract the SNI from the connection context and match it against pre-configured name mappings (exact or suffix match) to select the real server. Before measuring performance, we run preliminary tests to verify POND's correctness and functionality. Since POND's intervention is limited to the handshake phase and subsequent traffic continues to be routed by the kernel's native path, we focus our measurements on HTTPS flows with relatively smaller message sizes ranging from 1 KiB to 8 MiB.

We evaluate the packet-processing capability of POND and HAProxy using two metrics: queries per second (QPS) and throughput. QPS is reported by *wrk*, and throughput is calculated as the product of message size and QPS. In each experiment setup, we configure *wrk* to issue HTTPS requests to URLs corresponding to different payload sizes. To focus on throughput measurement, the name rules are limited to just 10 entries. Fig. 7a compares the QPS and throughput achieved by POND and HAProxy. For payloads ranging from 1 KiB to 8 MiB, POND outperforms HAProxy in both QPS and throughput. In particular, for small messages, POND delivers a higher packet-forwarding rate: at 16 KiB and 32 KiB, the QPS of POND exceeds that of HAProxy by 24.81 % and 33.74 %, respectively. For larger messages, POND achieves superior throughput: at 4 MiB and 8 MiB, its throughput is 10.79 % and 8.36 % higher than that of HAProxy. These improvements stem from the design of POND, in which handshaking, proxying, and name resolution are all performed in kernel space. During the experiments, we observed that HAProxy incurred an average of 10.9 K involuntary context switches per second. To be contrast, packets share the same socket buffer without context switches or copies into user-space in POND.

End-to-end Latency. We employed the same method as in the previous section to issue requests. For each message size, we run a three-minute test and recorded the traffic with tshark [31] for later analysis. Fig. 7b compares the 50th and 90th-percentile end-to-end latencies of POND and HAProxy across various message sizes. As shown, POND consistently achieves lower end-to-end latency than HAProxy regardless of message size. In particular, at 16 KiB and 32 KiB message sizes, POND reduces end-to-end latency by 31.75% and 33.92%, respectively, compared to HAProxy. Besides the extra overhead of context switches and memory copies discussed in the previous section, this improvement also stems from HAProxy's relatively longer code-processing path, for example, its linear-time ACL evaluation.

Scalability of Name-rule Queries. In addition to end-to-

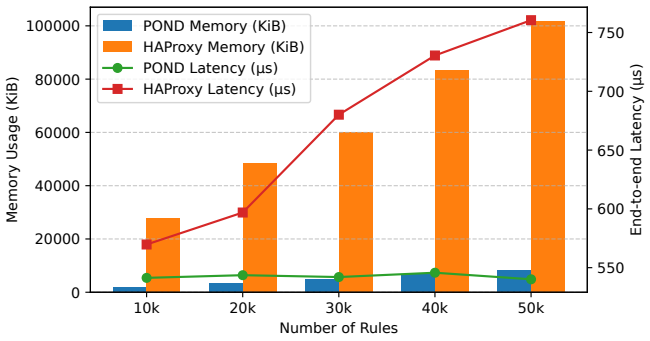


Fig. 8: Incremental memory footprint and average per-request end-to-end latency across varying rule-set sizes

end throughput and latency, another metric to measure the performance of the POND system is the scalability of the deferred name resolution. We use the Cisco Umbrella Popularity List [32] as our domain corpus in order to approximate the diversity and scale of real-world production traffic. From that list, we select a subset of fully qualified domains for exact-match rules and a subset of public suffixes for suffix-match rules. We then generate *wrk* client requests whose `Host` headers and URLs cycle through these domains, thus emulating clients accessing a wide range of services. To demonstrate the name-resolution scalability of the POND system, we run experiments on rule-set sizes of 10 K to 50 K entries.

In the POND system, name rules are stored in the in-kernel radix tree data structure proposed in Section IV-C. In HAProxy, the same rule sets are encoded as ACLs in its configuration file, using the `-i` option for case-insensitive exact matches and `-m end` for suffix matches. The real HTTP server remained the same as in our previous experiments, but in this evaluation it only served a 2 KiB static payload per request.

We measure two metrics: 1) the memory footprint of the name-rule data structure, and 2) the average end-to-end latency per request. For the radix tree of POND, we determined memory usage by reading the relevant slab entry in `/proc/slabinfo` (since each tree node is allocated from a dedicated `kmem_cache`). For the ACL set of HAProxy , we use a differential measurement: we subtract the resident memory of HAProxy with empty rules from its resident memory after loading the rule set. To measure latency, we run each test for three minutes, capture all packets with tshark, and then compute per-request average latency.

Fig. 8 plots both metrics as a function of rule-set size. The memory footprint of POND grows much more slowly than that of HAProxy. Although both radix-tree lookups and ACL-based evaluation shares the time complexity of $O(n)$ in the number of rules n, the per-rule overhead of HAProxy is significantly larger. In terms of request latency, HAProxy shows a roughly linear increase as the ACL set grows, whereas the average end-to-end latency of POND remains essentially flat. This is explained by the radix-tree lookup complexity of $\Omega(\log_r n + m)$, where m is the query length and the fan-out $r = 40$ yields a shallow tree (depth 4-5 in our experiments). The near-constant latency and modest memory growth together demonstrate that the deferred name resolution design of POND system achieves excellent scalability.

VI. Discussion

Name Preference. Although we only use HTTPS traffic in our experiments to evaluate the performance of POND, the POND architecture and the 4-way TCP-TLS joint-handshake proxy are applicable to all TLS-based traffic, including FTPS, PostgreSQL over SSL, and so forth. In fact, the same approach can be adapted to any TCP-based protocol whose handshake process carries name information. A typical example is SSH: the Protocol Version Exchange string (e.g., `SSH-2.0-OpenSSH_10.0`) sent by the client at the start of the connection can be leveraged to carry name information. Practically, this can be achieved by configuring the SSH client (e.g., using the `VersionAddendum` option in `ssh_config`) to append a suffix like `@target.svc`. POND can intercept this version string, extract the target name, and route the connection to the corresponding backend before the encrypted session begins. Furthermore, in cross-realm resource access scenarios, the use of encrypted Client Hello (ECH [33], [34]) for privacy consideration is not mandatory. Even if ECH is enabled, POND remains functional by routing based on the unencrypted Outer SNI.

Security Considerations. The introduction of the 4-way TCP-TLS joint handshake does not compromise the security model of TLS. It is crucial to note that POND acts solely as a transparent forwarding agent at the TCP layer; it does not terminate the TLS connection. POND only inspects the initial `Client Hello` packet to extract the SNI and subsequently relays all TLS packets between the client and the real server without modification. Since the cryptographic parameters are negotiated end-to-end, POND cannot decrypt or tamper with the encrypted application data as a man-in-the-middle, preserving the integrity of the TLS session. Furthermore, deploying the name resolution logic within the kernel requires rigorous safety guarantees. The POND radix tree is designed with strict bounds: the traversal depth is limited by the maximum length of a domain name, ensuring deterministic lookup time and preventing algorithmic denial-of-service attacks. Memory is managed via the kernel's slab allocator with proper isolation, and access to the configuration interface is restricted to users with `CAP_NET_ADMIN` privileges, thereby securing the system against unauthorized rule manipulation.

eBPF Acceleration. Some research has proposed re-implemented parts of the network functionalities with eBPF to boost performance, e.g. HyperCom [35], Tigger [36]. It is important to note, however, that before the TLS Client Hello packet arrives, the proxy should suspend the preceding TCP handshake, since the IP address of real server cannot yet be determined without name. For such stateful connection operations, eBPF/XDP on its own is insufficient: one also needs support in kernel-space or user-space components to

perform packet stealing, connection state maintenance, and packet issuing. Nevertheless, other studies (e.g., [21], [37], [38], [39]) have shown that the stateless parts of the proxying process can be offloaded to XDP by issuing SYN cookies there. Applying this idea to POND may offer further gains in proxy performance.

VII. Conclusion

In this paper, we propose POND, which resolves the issue of IP address space overlap using a name-based routing scheme. With POND, services in different intranets with overlapping IPs can be accessed consistently and simultaneously both within intranets and over the Internet. To support transparent name-based routing, we use the server name carried in the TLS SNI extension as the name for each TLS connection. A 4-way TCP-TLS joint handshake proxy mediates both the TCP and TLS handshakes and extracts the server name. We then perform deferred in-kernel name resolution at routing time using the POND radix tree, which supports both exact and suffix-based rule matching to redirect traffic to the intended real server. The entire process runs in kernel space. Thus, POND incurs no context switches or user-space memory copies. Our experimental results show that, across a range of message sizes, POND outperforms the user-space load balancer HAProxy in end-to-end throughput and latency, and scales well as the size of the name-rule set grows.

References

[1] A. W. Services, "Aws transit gateway," 2026, [Online; accessed 11-Jan-2026]. [Online]. Available: https://aws.amazon.com/transit-gateway/

[2] M. Azure, "Azure virtual wan," 2026, [Online; accessed 11-Jan-2026]. [Online]. Available: https://learn.microsoft.com/en-us/azure/virtual-wan/

[3] Y. Rekhter, B. Moskowitz, D. Karrenberg, G. J. de Groot, and E. Lear, "Address allocation for private internets," Tech. Rep., 1996.

[4] J. A. Donenfeld, "Wireguard: Next generation kernel network tunnel." in *NDSS*, 2017, pp. 1–12.

[5] P. Srisuresh and K. Egevang, "Traditional ip network address translator (traditional nat)," Tech. Rep., 2001.

[6] D. T. Narten, E. Gray, D. L. Black, L. Fang, L. Kreeger, and M. Napierala, "Problem Statement: Overlays for Network Virtualization," RFC 7364, Oct. 2014. [Online]. Available: https://www.rfc-editor.org/info/rfc7364

[7] J. Peterson, O. Kolkman, H. Tschofenig, and D. B. D. Aboba, "Architectural Considerations on Application Features in the DNS," RFC 6950, Oct. 2013. [Online]. Available: https://www.rfc-editor.org/info/rfc6950

[8] P. Francis and R. Gummadi, "Ipnl: A nat-extended internet architecture," in *Proceedings of the 2001 conference on Applications, technologies, architectures, and protocols for computer communications*, 2001, pp. 69–80.

[9] R. Moskowitz and M. Komu, "Host Identity Protocol Architecture," RFC 9063, Jul. 2021. [Online]. Available: https://www.rfc-editor.org/info/rfc9063

[10] HAProxy. (2025) The reliable, high performance tcp/http load balancer. [Online]. [Online]. Available: https://www.haproxy.org/

[11] nginx. (2025) nginx. [Online]. [Online]. Available: https://nginx.org/

[12] Google, "Google ipv6 statistics," 2026, [Online; accessed 11-Jan-2026]. [Online]. Available: https://www.google.com/intl/en/ipv6/statistics.html

[13] J. Weil, V. Kuarsingh, C. Donley, C. Liljenstolpe, and M. Azinger, "IANA-Reserved IPv4 Prefix for Shared Address Space," RFC 6598, Apr. 2012. [Online]. Available: https://www.rfc-editor.org/info/rfc6598

[14] F. Baker and M. Cullen, "IPv6-to-IPv6 Network Prefix Translation," RFC 6296, Jun. 2011. [Online]. Available: https://www.rfc-editor.org/info/rfc6296

[15] A. Gulbrandsen, P. Vixie, and L. Esibov, "A dns rr for specifying the location of services (dns srv)," Tech. Rep., 2000.

[16] M. Mahalingam, D. Dutt, K. Duda, P. Agarwal, L. Kreeger, T. Sridhar, M. Bursell, and C. Wright, "Virtual eXtensible Local Area Network (VXLAN): A Framework for Overlaying Virtualized Layer 2 Networks over Layer 3 Networks," RFC 7348, Aug. 2014. [Online]. Available: https://www.rfc-editor.org/info/rfc7348

[17] J. J. Garcia-Luna-Aceves, "Name-based content routing in information centric networks using distance information," in *Proceedings of the 1st ACM Conference on Information-Centric Networking*, 2014, pp. 7–16.

[18] D. S. E. Deering and B. Hinden, "Internet Protocol, Version 6 (IPv6) Specification," RFC 8200, Jul. 2017. [Online]. Available: https://www.rfc-editor.org/info/rfc8200

[19] A. Afanasyev, J. Burke, T. Refaei, L. Wang, B. Zhang, and L. Zhang, "A brief introduction to named data networking," in *MILCOM 2018-2018 IEEE Military Communications Conference (MILCOM)*. IEEE, 2018, pp. 1–6.

[20] X. Shi, L. He, J. Zhou, Y. Yang, and Y. Liu, "Miresga: Accelerating layer-7 load balancing with programmable switches," in *Proceedings of the ACM on Web Conference 2025*, 2025, pp. 2424–2434.

[21] S. Yoo, X. Chen, and J. Rexford, "{SmartCookie}: Blocking {Large-Scale}{SYN} floods with a {Split-Proxy} defense on programmable data planes," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 217–234.

[22] L. L. Peterson and B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2024.

[23] Google. (2025, Jul.) Https encryption on the web. [Online]. [Online]. Available: https://transparencyreport.google.com/https/overview

[24] S. Labs. (2025, Jun.) Ssl pulse. [Online]. [Online]. Available: https://www.ssllabs.com/ssl-pulse/

[25] D. Eastlake 3rd, "Rfc 6066: Transport layer security (tls) extensions: Extension definitions," 2011.

[26] J. Engelhardt and N. Bouliane, "Writing netfilter modules," *Revised, February*, vol. 7, 2011.

[27] P. McHardy. (2013, Aug.) netfilter: implement netfilter syn proxy. [Online]. [Online]. Available: https://lwn.net/Articles/563151/

[28] M. Husák, M. Čermák, T. Jirsík, and P. Čeleda, "Https traffic analysis and client identification using passive ssl/tls fingerprinting," *EURASIP Journal on Information Security*, vol. 2016, no. 1, p. 6, 2016.

[29] J. Althouse. (2025) Tls fingerprinting with ja3 and ja3s. [Online]. [Online]. Available: https://engineering.salesforce.com/tls-fingerprinting-with-ja3-and-ja3s-247362855967/

[30] W. Glozer, "wrk—a HTTP benchmarking tool," 2025, gitHub repository [Online]. [Online]. Available: https://github.com/wg/wrk

[31] "Wireshark," 2025, [Online]. [Online]. Available: https://www.wireshark.org/

[32] Cisco, "Cisco umbrella popularity list," 2025, [Online]. [Online]. Available: https://s3-us-west-1.amazonaws.com/umbrella-static/index.html

[33] E. Rescorla, K. Oku, N. Sullivan, and C. A. Wood, "TLS Encrypted Client Hello," Internet Engineering Task Force, Internet-Draft draft-ietf-tls-esni-25, Jun. 2025, work in Progress. [Online]. Available: https://datatracker.ietf.org/doc/draft-ietf-tls-esni/25/

[34] C. Huitema, "Issues and Requirements for Server Name Identification (SNI) Encryption in TLS," RFC 8744, Jul. 2020. [Online]. Available: https://www.rfc-editor.org/info/rfc8744

[35] B. Yang, D. Shen, H. Yang, L. Zhao, J. Liu, J. Cheng, and B. Wang, "Hypercom: Enabling high performance and composable data structures for software network functions with ebpf," in *IEEE INFOCOM 2025-IEEE Conference on Computer Communications*. IEEE, 2025, pp. 1–10.

[36] M. Butrovich, K. Ramanathan, J. Rollinson, W. S. Lim, W. Zhang, J. Sherry, and A. Pavlo, "Tigger: A database proxy that bounces with user-bypass," *Proceedings of the VLDB Endowment*, vol. 16, no. 11, pp. 3335–3348, 2023.

[37] M. Mikityanskiy. (2021, Jul.) Accelerating syn-proxy with xdp. [Online]. [Online]. Available: https://netdevconf.info/0x15/session.html?Accelerating-synproxy-with-XDP

[38] S. F. P. Penkov, E. Dumazet. (2020, Jul.) Issuing syn cookies in xdp. [Online]. [Online]. Available: https://netdevconf.info/0x14/session.html?talk-issuing-SYN-cookies-in-XDP

[39] K. Iwashima. (2023, Nov.) Syn proxy at scale with bpf. [Online]. [Online]. Available: https://lpc.events/event/17/contributions/1645/